

Relatório Técnico - Computational Thinking with Python

Projeto: HoloPass - Global Solution 2026 - Indústria Espacial

Arquivo principal: `holopass\_menu.py`

Integrante: Thiago Souza de Lima - RM 568732

1. Objetivo do programa

O programa apresenta um menu em Python para simular operações essenciais do HoloPass. Ele foi desenvolvido como arquivo único, sem bibliotecas externas, para facilitar a correção no Portal.

A solução conecta transporte público e tecnologia espacial ao usar GNSS, Haversine, saldo, recarga, NFC e cálculo de rota operacional em um fluxo simples de primeiro semestre.

2. Funcionalidades implementadas

O menu possui seis opções funcionais e retorna ao início após cada execução:

- Descrição do projeto em até cinco linhas.
- Detecção da estação mais próxima por coordenadas GNSS.
- Cálculo de rota usando todas as estações cadastradas no app web.
- Cálculo de linhas, baldeações, distância operacional, tempo e tarifa.
- Recarga de saldo.
- Pagamento NFC simulado com validação de saldo.
- Histórico e estatísticas de viagens.

3. Requisitos obrigatórios atendidos

- if/elif/else: usado para validação, saldo e origem/destino iguais.
- match-case: usado no controle principal das opções.
- while: mantém o menu ativo e repete entradas inválidas.
- for: percorre linhas, estações, arestas do grafo e histórico.
- listas: armazenam estações, transferências, fila de busca e histórico.
- strings: tratam nomes de linhas, estações, mensagens e moeda.
- funções: recebem parâmetros e retornam valores calculados.
- validação: impede letras em campos numéricos e opções fora de faixa.

O código usa apenas `math` e `sys`, ambas bibliotecas nativas do Python. Não há instalação via `pip`.

4. Uso de recursos extras com moderação

O programa usa poucos recursos além dos pilares obrigatórios:

- ``try/except``: aparece apenas para manter compatibilidade de saída UTF-8 no terminal e validar números decimais digitados pelo usuário.
- Dicionários: organizam linhas, estações e dados de rota de forma legível.

Esses recursos não substituem os conceitos avaliados. Eles apenas deixam a solução mais clara e estável, mantendo o escopo adequado para o primeiro semestre.

5. Funções principais

- ``haversine_km(lat1, lon1, lat2, lon2)``: retorna distância real em quilômetros para GNSS.
- ``construir_grafo()``: cria trechos de linha, baldeações e corredores.
- ``calcular_rota(origem, destino)``: retorna linhas, estações, baldeações, distância, tempo e tarifa.
- ``escolher_estacao(titulo)``: lista todas as estações e valida a escolha.
- ``brl(valor)``: formata valores no padrão brasileiro.

6. Evidências esperadas de execução

Ao rodar o programa, o avaliador deve observar:

- O menu reaparece após cada opção.
- Entradas inválidas são recusadas e pedidas novamente.
- A opção de rota lista a rede completa de metrô/CPTM cadastrada no projeto.
- A rota Sé -> Paulista passa por baldeação e não fica em apenas seis minutos.
- A tarifa exibida e debitada é R\$ 5,40.
- Recarga altera o saldo.
- Pagamento NFC aprova com saldo suficiente e nega com saldo insuficiente.

Comando:

```
python computational-thinking/holopass_menu.py
```

Não é necessário instalar nenhuma dependência adicional.

7. Observações para correção

O programa evita dependências externas e bibliotecas instaladas por ``pip``. O algoritmo de rota é implementado com listas, dicionários, laços e funções, mantendo o código compreensível para primeiro semestre.

O foco está nos pilares da avaliação: decisão, repetição, listas, strings, funções, validação e usabilidade.

8. Conclusão

O protótipo atende ao pedido de Computational Thinking porque transforma a ideia do HoloPass em uma simulação funcional. Ele mostra como GNSS, saldo, rota, baldeação e NFC podem ser representados com lógica estruturada em Python.